Practice 10 — Ship an Image: Registry, Server, Release, Rollback

Objective: take last week's Flask app, push it to a **registry**, run it on a **server** that pulls the image (never builds it), release **v2**, and **roll back** to v1. Track A does it with **Amazon ECR + EC2** in Learner Lab; Track B does the same on your laptop with a local registry. These are the skills from today's lecture.

Timebox: ~55–60 min, in class. Track A spends ~5 minutes waiting for the instance to boot: build and push while you wait. If you're stuck on one step for more than 10 minutes, ask instead of pushing through alone.

> **How you're checked:** this practice is done in class and **checked orally** at the end. There's no written report. Keep your terminal (and console, on Track A) open, be ready to **show** each item in the Definition of done and **explain** what each step does. The instructor will ask you a few of the questions under "Be ready to explain".

> **Two tracks.** Do **Track A** if you have AWS Academy Learner Lab. Do **Track B** if you don't, or if Learner Lab blocks ECR or the instance profile (`AccessDenied`). Both start with **Part 0**. Tell the instructor which track you did.

> **Before class (at home):** Docker working (`docker run hello-world`), and pull the images once: `docker pull python:3.12-slim` and `docker pull registry:2.8.3` .

Definition of done (things you can show)

□ Your **Dockerfile** with a non-root `USER` , and `whoami` inside the running container printing `appuser`
□ The image **in the registry** with tags `v1` and `v2` (Track A: `aws ecr list-images` ; Track B: `/v2/lab10-app/tags/list`)
□ The app answering **from the "server"**: Track A: `curl` to the instance's **public IP**; Track B: the container pulled from the registry on `localhost:8080`
□ Track A: proof there are **no access keys on the instance** — it pulled from ECR with its **instance profile**
□ `/version` showing **v2** after the release and **v1** after the rollback
□ Tag vs **digest** for v1 and v2, and which digest the server runs
□ The app **crashing** (`POST /crash`) and Docker **restarting** it (`--restart unless-stopped`)
□ Cleanup done and shown (Track A: instance, SG and ECR repository deleted, lab ended)

Part 0 — The app, with a version (everyone, ~10 min)

Make a folder `lab10` with these files. It's last week's app plus `/version` (read from an env var) and `/crash` (to test restart policies).

`app.py`:

```
import os, socket
from flask import Flask

app = Flask(__name__)
DATA = os.environ.get("DATA_DIR", "/data")
VERSION = os.environ.get("APP_VERSION", "dev")

@app.get("/")
def index():
    os.makedirs(DATA, exist_ok=True)
    return {"host": socket.gethostname(), "files": sorted(os.listdir(DATA))}

@app.get("/version")
def version():
    return {"version": VERSION, "message": "Hello from lab10"}

@app.post("/notes/<name>")
def add_note(name):
    os.makedirs(DATA, exist_ok=True)
    with open(os.path.join(DATA, f"{name}.txt"), "w") as f:
        f.write(f"note {name}\n")
    return {"saved": f"{name}.txt"}, 201

@app.post("/crash")
def crash():
    os._exit(1)          # simulate the app dying

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

`requirements.txt`:

```
flask==3.1.3
```

`.dockerignore`:

```
.venv/
__pycache__/
.env
*.pem
```

`Dockerfile` — fill in the TODOs (all on the lecture slides):

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ARG APP_VERSION=dev
ENV APP_VERSION=$APP_VERSION DATA_DIR=/data
# TODO 1: create a user "appuser" with a home folder, create /data
#         and make appuser its owner (one RUN line, joined with &&)
# TODO 2: switch to that user
EXPOSE 8000
CMD ["python", "app.py"]
```

`ARG` is a **build-time** value: `--build-arg APP_VERSION=v1` bakes the version label into the image's `ENV`. Nothing secret goes in there.

Quick local test:

```
docker build --build-arg APP_VERSION=v1 -t lab10-app:v1 .
docker run --rm -d --name test -p 8001:8000 lab10-app:v1
curl -s localhost:8001/version     # {"message":"Hello from lab10","version":"v1"}
docker exec test whoami            # appuser
docker rm -f test
```

Track A — Amazon ECR + EC2 (AWS Academy Learner Lab)

> **Using AWS Academy Learner Lab**
>
> 1. **Modules** → **Launch AWS Academy Learner Lab** → **Start Lab**, wait for the **green** dot.
> 2. Region **`us-east-1`**. Per the Learner Lab Readme, only `us-east-1` and `us-west-2` are allowed.
> 3. **AWS Details** → **AWS CLI** → **Show**: put the three values into `~/.aws/credentials` under `[default]` (they are temporary; copy them again each session). **AWS Details** → **Download PEM** gives you `labsuser.pem` for the key pair **`vockey`**; `chmod 400 labsuser.pem`.
> 4. You can't create IAM roles; the instance uses the provided **`LabInstanceProfile`**. If ECR or the instance profile gives `AccessDenied`, switch to **Track B**.

> 5. When you're done: Cleanup, then **End Lab**.

A1. ECR repository and login

```
export AWS_DEFAULT_REGION=us-east-1
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
REGISTRY=$ACCOUNT.dkr.ecr.us-east-1.amazonaws.com
echo "$REGISTRY"

aws ecr create-repository --repository-name lab10-app \
  --image-scanning-configuration scanOnPush=true \
  --query repository.repositoryUri --output text

aws ecr get-login-password | docker login --username AWS --password-stdin "$REGISTRY"
# Login Succeeded
```

A2. Build for the instance's CPU and push v1

The instance is `x86_64`. On an **Apple Silicon Mac** you must build for `linux/amd64`; on Windows/Intel it's the default anyway, the flag does no harm.

```
docker buildx build --platform linux/amd64 \
  --build-arg APP_VERSION=v1 -t "$REGISTRY/lab10-app:v1" --push .

aws ecr list-images --repository-name lab10-app
aws ecr describe-image-scan-findings --repository-name lab10-app \
  --image-id imageTag=v1 --query 'imageScanFindings.findingSeverityCounts'
```

The scan may take a minute, and depending on the account's scanning settings it may return no counts at all. That's fine; note what you got.

A3. Security group and user data

```
VPC=$(aws ec2 describe-vpcs --filters Name=is-default,Values=true \
  --query 'Vpcs[0].VpcId' --output text)
SG=$(aws ec2 create-security-group --group-name lab10-web \
  --description "lab10 app" --vpc-id "$VPC" --query GroupId --output text)
MYIP=$(curl -s https://checkip.amazonaws.com)
aws ec2 authorize-security-group-ingress --group-id "$SG" --protocol tcp --port 80 --cidr
0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id "$SG" --protocol tcp --port 22 --cidr
"$MYIP/32"
```

Create `user-data.sh`. The unquoted `EOF` makes your shell fill in `$REGISTRY` **now**, so the instance gets the real address:

```
cat > user-data.sh <<EOF
#!/bin/bash
dnf install -y docker
systemctl enable --now docker
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin $REGISTRY
docker run -d --name app --restart unless-stopped -p 80:8000 $REGISTRY/lab10-app:v1
EOF
cat user-data.sh          # check the registry address is filled in
```

There are **no keys** in this file. On the instance, the AWS CLI gets temporary credentials from the **instance profile**.

A4. Launch and test

```
IID=$(aws ec2 run-instances \
  --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --instance-type t3.micro --key-name vockey \
  --security-group-ids "$SG" \
  --iam-instance-profile Name=LabInstanceProfile \
  --user-data file://user-data.sh \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lab10-app}]' \
  --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-running --instance-ids "$IID"
IP=$(aws ec2 describe-instances --instance-ids "$IID" \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
echo "$IP"
```

Give it 1–2 minutes (Docker install + image pull), then:

```
curl -s "http://$IP/version"      # {"message":"Hello from lab10","version":"v1"}
curl -s "http://$IP/"             # host = the container's ID
```

SSH in and look around:

```
ssh -i labsuser.pem ec2-user@"$IP"
sudo docker ps                      # app, Up …, 0.0.0.0:80->8000/tcp
sudo docker exec app whoami         # appuser
ls ~/.aws 2>/dev/null; env | grep AWS_    # nothing: no keys on this server
TOKEN=$(curl -s -X PUT http://169.254.169.254/latest/api/token -H "X-aws-ec2-metadata-token-ttl-
seconds: 60")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/iam/info
#   ... "InstanceProfileArn": "...instance-profile/LabInstanceProfile" ...
aws sts get-caller-identity         # an assumed-role ARN: the instance's role
```

If `curl` gets nothing, read `sudo tail -n 30 /var/log/cloud-init-output.log`.

A5. Release v2

On your **laptop**, change the message in `app.py` (e.g. `"Hello from lab10, now with v2"`), then:

```
docker buildx build --platform linux/amd64 \
  --build-arg APP_VERSION=v2 -t "$REGISTRY/lab10-app:v2" --push .
aws ecr list-images --repository-name lab10-app      # v1 and v2
```

On the **instance** (SSH):

```
REGISTRY=<your account>.dkr.ecr.us-east-1.amazonaws.com
aws ecr get-login-password --region us-east-1 | sudo docker login --username AWS --password-stdin
$REGISTRY
sudo docker pull $REGISTRY/lab10-app:v2
sudo docker rm -f app
sudo docker run -d --name app --restart unless-stopped -p 80:8000 $REGISTRY/lab10-app:v2
curl -s localhost/version            # version v2, new message
```

A6. Roll back, digests, crash test

Still on the instance:

```
sudo docker rm -f app
sudo docker run -d --name app --restart unless-stopped -p 80:8000 $REGISTRY/lab10-app:v1
curl -s localhost/version            # back to v1 — in seconds, image already cached

sudo docker image inspect --format '{{index .RepoDigests 0}}' $REGISTRY/lab10-app:v1
$REGISTRY/lab10-app:v2
sudo docker inspect --format '{{.Config.Image}} {{.Image}}' app

curl -s -X POST localhost/crash; echo " exit=$?"   # no reply, exit=52: the app died
sleep 3
sudo docker inspect --format '{{.State.Status}} restarts={{.RestartCount}}' app
#   running restarts=1
```

Compare the digests with `aws ecr list-images --repository-name lab10-app` on your laptop: same `sha256:…` values.

A7. Cleanup (required: the instance bills per hour)

```
aws ec2 terminate-instances --instance-ids "$IID"
aws ec2 wait instance-terminated --instance-ids "$IID"
aws ec2 delete-security-group --group-id "$SG"
aws ecr delete-repository --repository-name lab10-app --force
docker rmi "$REGISTRY/lab10-app:v1" "$REGISTRY/lab10-app:v2" lab10-app:v1 2>/dev/null
```

Then **End Lab**.

Track B — local registry (no AWS)

AWS (Track A)	Track B
ECR repository	A local registry: `registry:2.8.3` on `localhost:5001`
`aws ecr get-login-password | docker login`	Not needed (no auth on the local registry)
`$REGISTRY/lab10-app:v1`	`localhost:5001/lab10-app:v1`
EC2 "server" pulling the image	Delete local copies, `docker pull`, run on port 8080
Instance profile instead of keys	(no equivalent: discuss it)

Port **5001**, not 5000: on macOS, port 5000 belongs to AirPlay.

B1. Start the registry, push v1

```
docker run -d --name registry --restart unless-stopped -p 5001:5000 registry:2.8.3
curl -s localhost:5001/v2/_catalog                # {"repositories":[]}

R=localhost:5001/lab10-app
docker build --build-arg APP_VERSION=v1 -t $R:v1 .
docker push $R:v1                                 # v1: digest: sha256:… size: …
curl -s localhost:5001/v2/_catalog                # {"repositories":["lab10-app"]}
curl -s localhost:5001/v2/lab10-app/tags/list     # {"name":"lab10-app","tags":["v1"]}
```

B2. Be the server: pull, don't build

Delete every local copy, so the image **must** come from the registry:

```
docker rmi $R:v1 lab10-app:v1 2>/dev/null
docker pull $R:v1                                 # Downloaded newer image for …
docker run -d --name prod --restart unless-stopped -p 8080:8000 $R:v1
curl -s localhost:8080/version                    # {"message":"Hello from lab10","version":"v1"}
docker exec prod whoami                           # appuser
```

B3. Release v2, roll back

Change the message in `app.py` (e.g. `"Hello from lab10, now with v2"`):

```
docker build --build-arg APP_VERSION=v2 -t $R:v2 .     # pip install: CACHED
docker push $R:v2
curl -s localhost:5001/v2/lab10-app/tags/list          # tags v1 and v2

docker pull $R:v2
docker rm -f prod
docker run -d --name prod --restart unless-stopped -p 8080:8000 $R:v2
curl -s localhost:8080/version                         # v2, new message

docker rm -f prod                                      # rollback
docker run -d --name prod --restart unless-stopped -p 8080:8000 $R:v1
curl -s localhost:8080/version                         # v1 again
```

B4. Tags, digests and `latest`

```
docker image inspect --format '{{index .RepoDigests 0}}' $R:v1 $R:v2
docker inspect --format '{{.Config.Image}} {{.Image}}' prod   # the image prod runs

docker tag $R:v1 $R:latest && docker push $R:latest   # latest = v1's digest
docker tag $R:v2 $R:latest && docker push $R:latest   # latest = v2's digest now
```

The two `latest` pushes print **different** digests: the same name now points to different content. `v1` and `v2` never move.

Pull by digest (copy one from the output above):

```
docker run --rm -d --name bydigest -p 8081:8000 "$R@sha256:<v1 digest>"
curl -s localhost:8081/version                    # v1
docker rm -f bydigest
```

B5. Crash test

```
curl -s -X POST localhost:8080/crash; echo " exit=$?"   # no reply, exit=52: the app died
sleep 3
docker inspect --format '{{.State.Status}} restarts={{.RestartCount}}' prod
#   running restarts=1

docker run -d --name nopolicy -p 8082:8000 $R:v1   # no --restart
curl -s -X POST localhost:8082/crash; sleep 3
docker inspect --format '{{.State.Status}} exit={{.State.ExitCode}}' nopolicy
#   exited exit=1
docker rm -f nopolicy
```

B6. Cleanup

```
docker rm -f prod registry bydigest nopolicy 2>/dev/null
docker rmi $R:v1 $R:v2 $R:latest lab10-app:v1 2>/dev/null
docker ps -a | grep -E 'prod|registry'            # nothing left
```

Stretch (optional, both tracks)

- **Architectures:** on an Apple Silicon Mac, build both and compare: `docker buildx build --platform linux/amd64 -t lab10-app:v1-amd64 --load .`, then `docker image inspect --format '{{.Os}}/{{.Architecture}}' lab10-app:v1-amd64 lab10-app:v1` (`linux/amd64` vs `linux/arm64`). Explain what would happen on a `t3.micro` with the arm64 one.
- **Track A:** add an ECR lifecycle policy that keeps only the 5 newest images (`aws ecr put-lifecycle-policy`, JSON from the slides).

Be ready to explain

1. Registry, repository, tag, digest: what is each one? Which of them can change, and which can't?
2. Why shouldn't you deploy `latest`? What did your two `latest` pushes show?
3. Track A: how did the instance log in to ECR **without** any keys on it? Why is that better than copying your access keys there?
4. Why `--platform linux/amd64` on a Mac? What error would you see on the `t3.micro` without it?
5. `APP_VERSION` is an env var. Why configure the app through env vars instead of editing the code for each environment?
6. What does `--restart unless-stopped` do, and what happened with and without it after `/crash`?
7. Why was the rollback so fast? What would you roll back **to** if every release were tagged `latest`?
8. One EC2 instance with Docker vs ECS on Fargate: what would ECS do for you that you did by hand today?

Common mistakes

- **`no basic auth credentials` (Track A).** Not logged in to ECR, or the 12-hour token expired. Run `aws ecr get-login-password | docker login …` again (on the instance too, before pulling `v2`).
- **`exec format error` in `docker logs` on EC2.** You pushed an arm64 image from a Mac. Rebuild with `--platform linux/amd64` and push again.
- **`curl http://$IP` times out.** Port 80 missing in the security group, or user data hasn't finished: wait a minute, then read `/var/log/cloud-init-output.log` on the instance.
- **`Unable to locate credentials` on the instance.** The instance was launched without `--iam-instance-profile`. Terminate it and launch again with it — don't copy keys onto the server.
- **The server still shows the old version.** You reused a tag and didn't `docker pull`; `docker run` uses the cached image. Use a new tag.
- **`server gave HTTP response to HTTPS client` (Track B).** You used a host name other than `localhost` for the registry. Docker only allows plain HTTP to `localhost` registries by default.
- **`port is already allocated`.** An old container still holds 8080, 8001 or 5001: `docker ps -a`, then `docker rm -f <name>`.
- **`Permission denied` writing to `/data`.** You switched to `USER appuser` but didn't create `/data` and `chown` it first.
- **Track A: the bill.** A forgotten instance runs all night. Terminate it, delete the repository, **End Lab**.